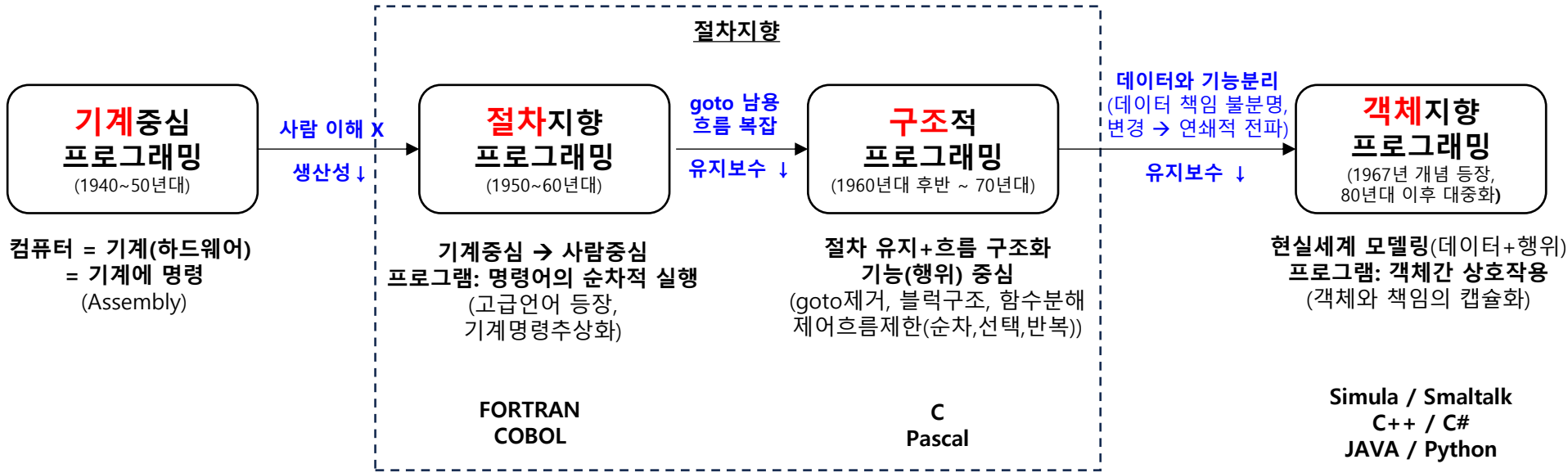


# 프로그래밍 패러다임의 변화

소프트웨어 규모/복잡도 증가 -> 생산성과 유지보수성의 문제



```
mov eax, 5      ; eax 레지스터에 5 저장  
mov ebx, 3      ; ebx 레지스터에 3 저장  
add eax, ebx    ;  $eax = eax + ebx$ 
```

## 패러다임의 변화: 절차지향 vs 객체지향

| 구분             | 절차 지향                                                        | 객체 지향                                                              |
|----------------|--------------------------------------------------------------|--------------------------------------------------------------------|
| 개념             | 함수를 중심으로 프로그램을 절차적으로 구성                                      | 데이터(상태)와 기능(행위)을 하나의 객체로 캡슐화하여 현실 세계를 모델링                          |
| 핵심사고           | 알고리즘(How to do)을 순차적으로 표현                                    | 책임 중심 (What to do)<br>(객체 간 메시지 전달과 책임 분배 중심)                      |
| 모델링 방식         | 프로세스/작업 흐름 중심 모델링                                            | 현실 세계의 개체(Entity)를 객체로 직접 매핑하여 모델링                                 |
| 데이터와<br>기능의 관계 | 데이터와 기능(함수)이 분리됨<br>(데이터는 함수 외부에 존재하며 여러 함수에서 접근)            | 데이터와 행위가 하나의 객체로 캡슐화되어 외부로부터 보호됨                                   |
| 재사용 단위         | 함수 단위 재사용                                                    | 클래스/객체 단위 (상속, 합성(Composition), 다형성 활용)                            |
| 변경/유지보수        | 전역 데이터 수정 → 전체 영향<br>(대규모 프로젝트에 취약)                          | 캡슐화화 정보은닉으로 변경 영향 범위 최소화<br>(대규모 시스템에 유리)                          |
| 장점             | 단순하고 직관적, 실행속도 빠름<br>(소규모 프로그램에 적합)                          | 코드 재사용성/확장성/유지보수성 높음<br>(대규모, 복잡한 시스템에 적합)                         |
| 단점             | 코드 규모 커질 수록 관리 어려움<br>데이터 무결성 보장 어려움, 코드 중복                  | 초기 설계·학습 비용 높음<br>런타임 오버헤드 약간 존재                                   |
| 구현 언어          | 순수 절차지향: C, Pascal, Fortran<br>현대 언어에서도 절차 스타일 가능 (Python 등) | 순수 OOP: Simula(기원), Smaltalk<br>대중화: C++, Java, C#, Python, Ruby 등 |